Spectrum Sensing in Cognitive Radio Using Cyclostationary Features

CSE400 - Fundamentals of Probability in Computing
Section 1
Group 11

Project Overview

Problem Statement

- Detect presence of primary OFDM user
- Work under unknown noise variance
- Ensure controlled false alarm probability

Core Idea of Paper

- Use cyclostationary structure of OFDM
- Combine multiple cyclic frequencies
- Derive analytical false alarm expression

Milestone 3 Goal

Previous milestones

• Understanding the probabilistic detection model.

• Understanding cyclostationary features

• Understanding noise uncertainty

Goal

•Verify theoretical detector behaviour using simulation

Milestone 3 objective

•Implement the entire detector pipeline

Tasks performed

• OFDM signal simulation

• Noise simulation

• Cyclostationary feature extraction

• Test statistic computation

• Monte Carlo evaluation

Detection Problem

Binary hypothesis testing model

- H0:x[n]=w[n]

Noise only

- H1:x[n]=s[n]+w[n]

Detector task

Decide whether the primary signal exists.

Signal + Noise

Where

- s[n] = OFDM signal
- w[n] = Gaussian noise
- x[n] = received signal

Why Cyclostationary Detection

Common sensing methods

- Matched Filter
- Energy Detection
- Cyclostationary Detection

Cyclostationary detection

- Exploits periodic signal structure
- Robust to unknown noise power
- Suitable for OFDM signals

Energy detector limitation

- Requires accurate noise variance
- Very sensitive to noise uncertainty

OFDM Cyclostationary Property

- **OFDM signals contain a cyclic prefix**
- **This introduces periodic correlations**

Cyclic autocorrelation

- $R_{x\alpha}(\tau)$

OFDM signal

- $R_{x\alpha}(\tau) \neq 0$

Noise

- $R_{w\alpha}(\tau) \approx 0$

Detector identifies signal presence using this difference.

Multi-Cycle Detector

Test statistic

- Tmc = Signal CAC energy / Reference CAC energy

Decision rule

- $Tmc \geq \lambda \rightarrow H1$
- $Tmc < \lambda \rightarrow H0$

Threshold chosen to maintain

- Pf=0.1

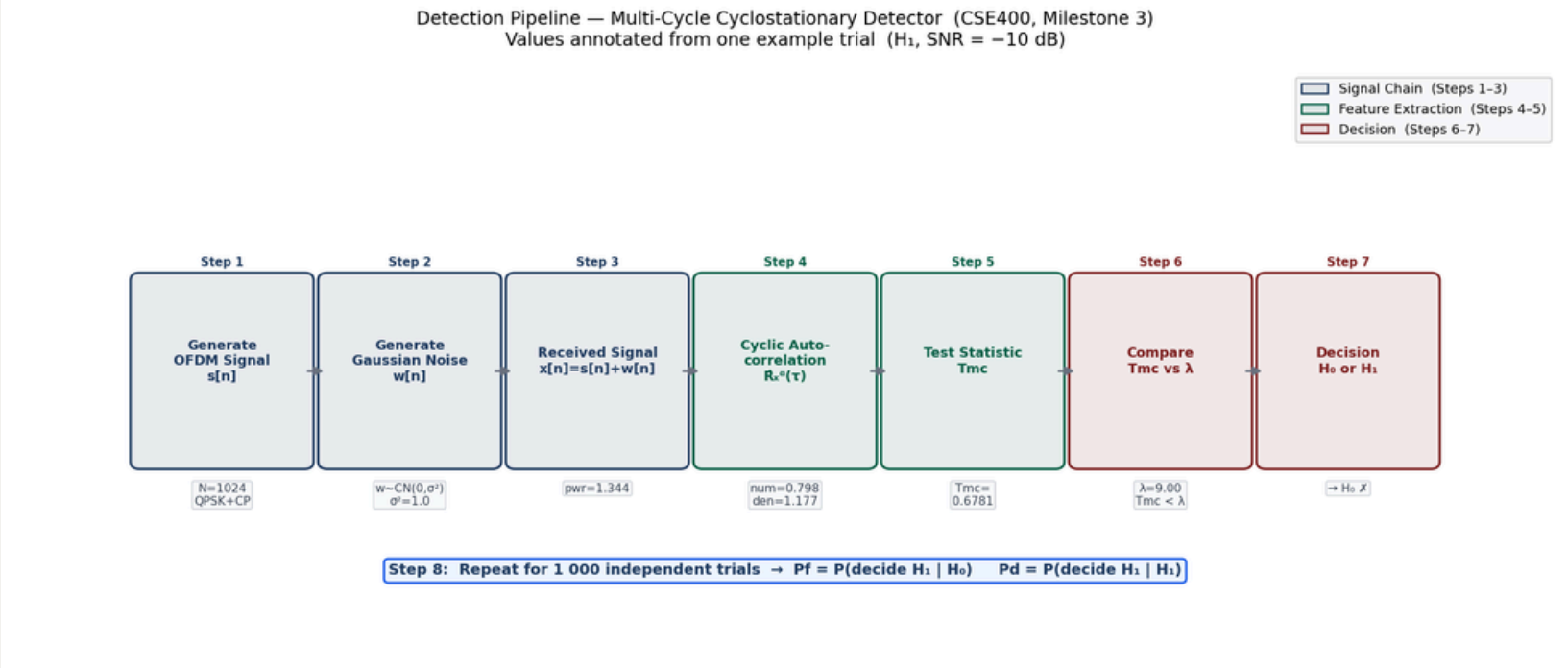
Multi-cycle approach

- Uses several cyclic frequencies simultaneously.

Detection Pipeline

Pipeline stages

1. Generate OFDM signal
2. Generate noise samples
3. Create received signal
4. Extract cyclostationary features
5. Compute detector statistic
6. Compare with threshold
7. Make decision



Simulation repeated using Monte Carlo trials.

Simulation Configuration

OFDM parameters

- FFT size = 64
- CP length = 16
- Modulation = QPSK

Detector parameters

- Cyclic frequencies
- [−2,−1,0,1,2]

Simulation settings

- Samples = 1024
- Trials = 1000
- SNR range = -20 dB to 0 dB

Target false alarm probability

- Pf=0.1

Code Architecture

Project Code Modules

- signal_generator.py
- noise_generator.py
- cyclostationary_features.py
- detector.py
- metrics.py
- run_simulation.py

Responsibilities

Signal generator

- Creates OFDM symbols

Noise generator

- Creates Gaussian noise

Feature extractor

- Computes cyclic autocorrelation

Detector

- Computes test statistic

Metrics

- Evaluates Pd and Pf

OFDM Signal Generation (Code)

File

- signal_generator.py

Main steps

- Generate QPSK symbols
- Apply IFFT
- Add cyclic prefix

Output

- Discrete OFDM signal samples s[n]

```python
# Generate all OFDM symbols
signal_parts = []
for _ in range(num_symbols):
    data = generate_qpsk_symbols(fft_size, rng)
    symbol = generate_ofdm_symbol(fft_size, cp_length, data)
    signal_parts.append(symbol)

full_signal = np.concatenate(signal_parts)

# Trim or extend to match required total_samples
if len(full_signal) >= total_samples:
    signal = full_signal[:total_samples]
else:
    # Tile the signal to fill required length
    repeats = int(np.ceil(total_samples / len(full_signal)))
    signal = np.tile(full_signal, repeats)[:total_samples]

# Normalize to desired power
current_power = np.mean(np.abs(signal) ** 2)
if current_power > 0:
    signal = signal * np.sqrt(signal_power / current_power)

return signal
```

Noise Generation

File

- noise_generator.py

Noise Model

- w[n]~CN(0,σ2)

Noise uncertainty

- Noise variance randomly perturbed.

```
if rng is None:
    rng = np.random.default_rng()

std_per_component = np.sqrt(variance / 2.0)
real_part = rng.normal(0.0, std_per_component, size=num_samples)
imag_part = rng.normal(0.0, std_per_component, size=num_samples)
return real_part + 1j * imag_part
```

Received Signal

Signal model

- x[n]=s[n]+w[n]

Top plot

- OFDM signal waveform

Middle plot

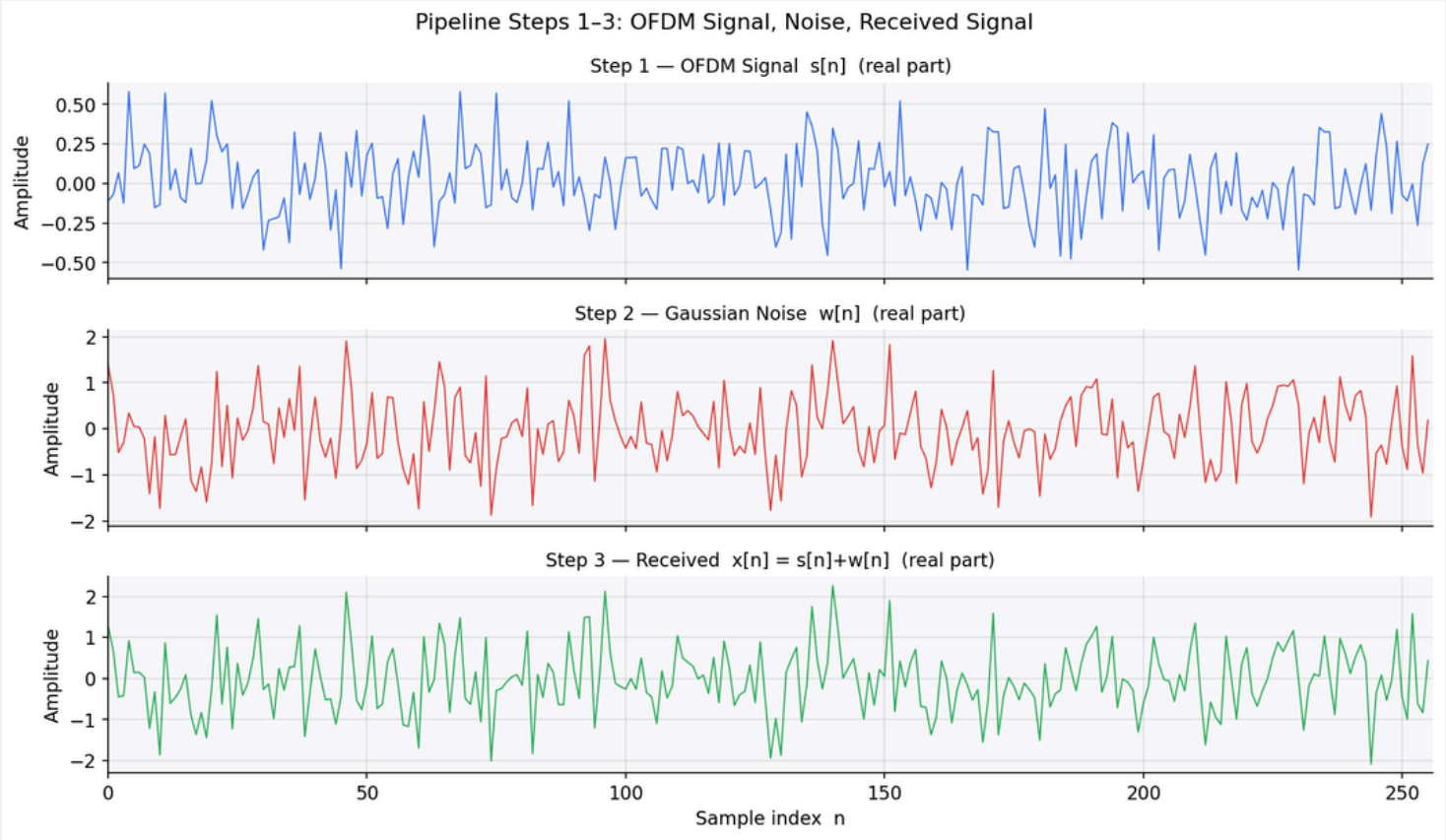
- Gaussian noise

Bottom plot

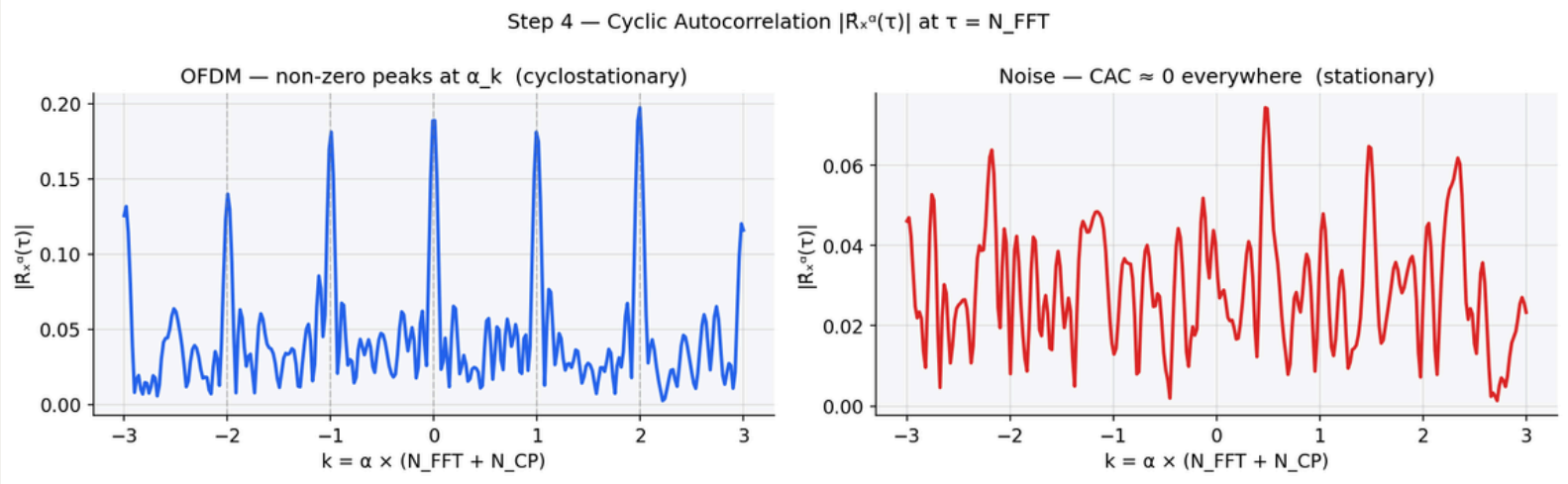
- Received signal after adding noise

Observation

- Noise dominates signal at low SNR.



Cyclostationary Feature Extraction



File

- cyclostationary_features.py

Conclusion

- Cyclostationary features reveal hidden signal structure.

Left plot - OFDM signal

- Clear peaks appear at specific cyclic frequencies.

Right plot - Noise signal

- No consistent peaks.

Ahmedabad University

Test Statistic Distribution

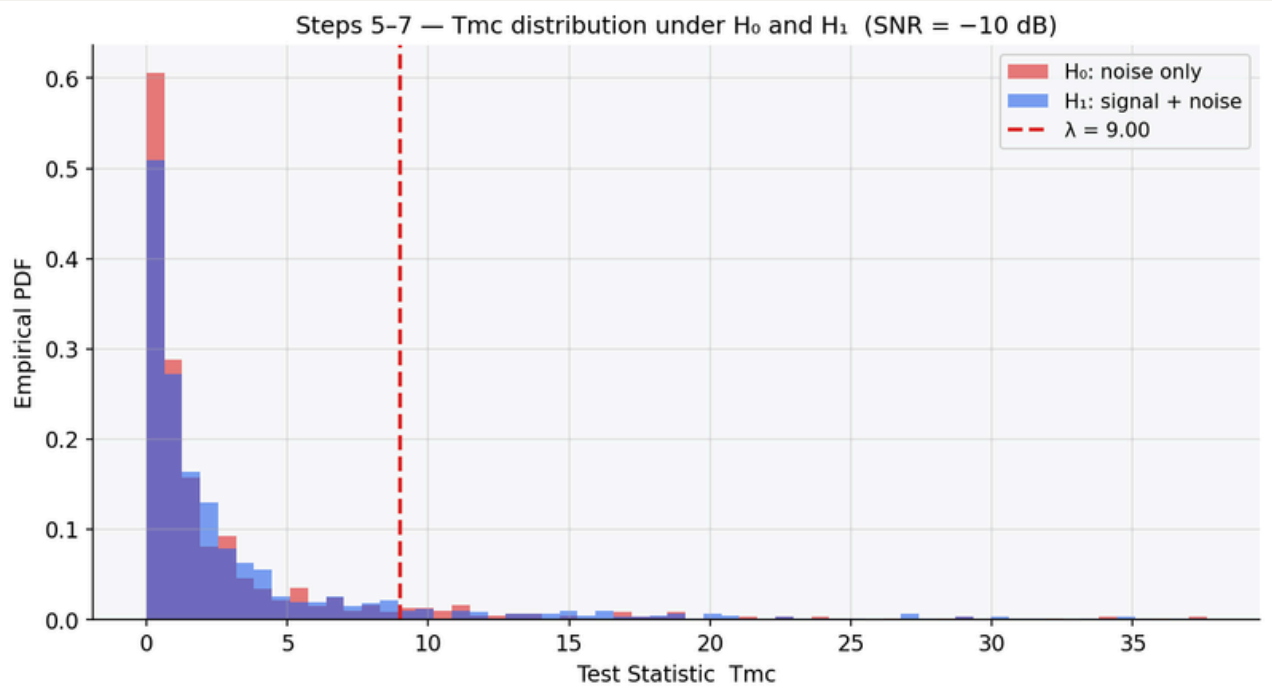
Chart interpretation:

Histogram shows distribution of test statistic.

Red histogram → Noise only (H0)

Blue histogram → Signal + noise (H1)

Vertical dashed line → Detection threshold λ



Observation:

Signal distribution shifts to higher values.

Detector separates the two hypotheses.

False Alarm Probability

Chart Interpretation:

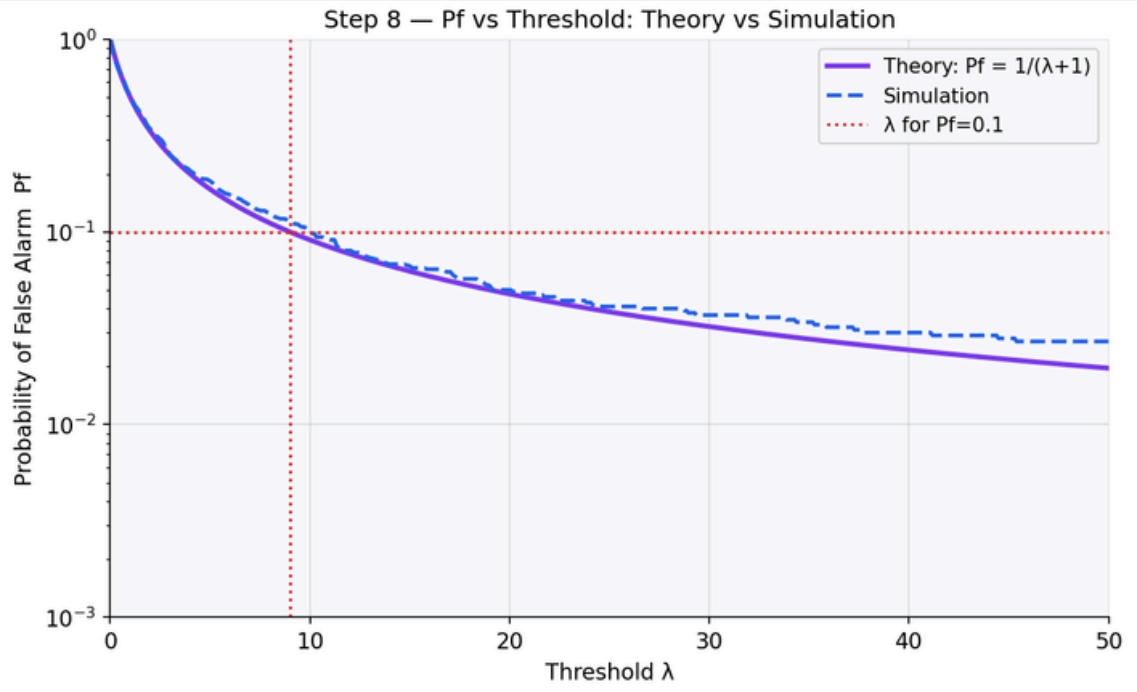
Graph shows relation between threshold λ and false alarm probability.

Theoretical result:

$$P_f = \frac{1}{\lambda + 1}$$

Observation:

Simulation curve closely matches theory. Detector behaviour follows theoretical model.



Ahmedabad University

Detection Probability vs SNR

Chart Interpretation

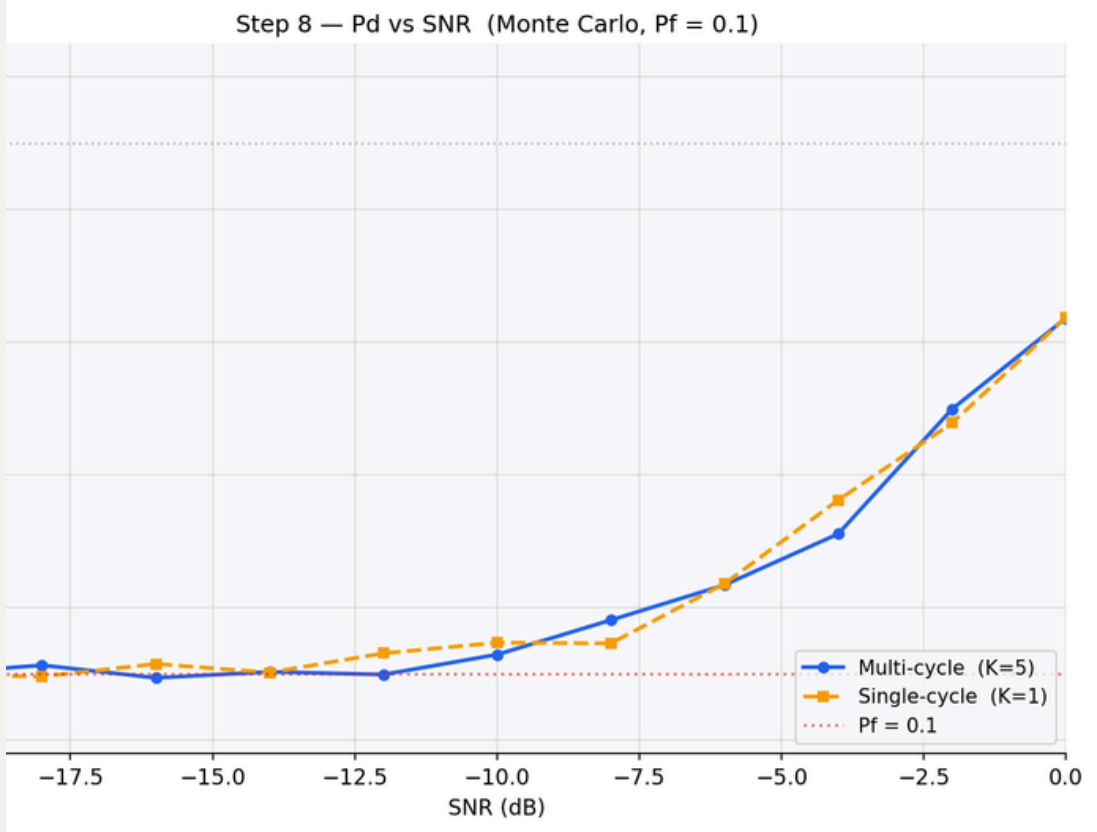
➔ Probability of detection increases as SNR increases.

➔ Low SNR region → Signal buried in noise.

➔ High SNR region → Signal structure easier to detect.

Comparison

Multi-cycle detector outperforms single-cycle detector. More cyclic frequency information used.



Key Observations

Current Assumptions

- ➔ Cyclostationary features distinguish signal and noise
- ➔ Multi-cycle detector improves performance
- ➔ Theoretical false alarm formula verified
- ➔ Detection improves with increasing SNR

Monte Carlo simulation confirms expected behaviour.

Limitations

Simplifications used

- ➔ Ideal OFDM signal model
- ➔ Independent noise samples
- ➔ No multipath fading

Future improvements

- ➔ Channel models
- ➔ Interference modelling
- ➔ Real signal datasets

Thank you!